RASP
Finance & People

The money and workforce specialist. Explains the accounting truth, the cash position, what is already committed, and the payroll consequence behind an operating decision.

Who this is for: product, engineering, operations and anyone who has to explain XELOR to somebody else.
What it covers: the work RASP reasons about, the rules it cannot break, the exact tools it can call, and what is honestly not built yet.
Truth standard: the repository as it stands on 7 August 2026.

RASP

01 · AT A GLANCE

What RASP is for

A role with a fixed tool list, not a general assistant

In one sentence

Explains the accounting truth, the cash position, what is already committed, and the payroll consequence behind an operating decision.

4 business areas it speaks for	**6** tools it can actually call	**5** capability families allowed	**0** agents it may delegate to

What reaches it

Journals, receivables and receipts · Order, purchasing and stock pressure · Budget commitments and claims · Attendance and payroll outcomes

What it hands back

The accounting position, as posted · A scenario clearly labelled as a scenario · A collection priority order · A finance work item or a reviewable draft

Capability families it is allowed to touch

accounts. · finance. · expenditure. · hrm. · agent.

Ownership and authority are not the same thing

The areas above are where RASP is the right specialist to ask. The tool table on page 8 is the much smaller set it can invoke at runtime. Owning a business area does not grant runtime power over it — and for ONYX, the supervisor, the gap between the two is the widest in the system.

What sits behind RASP

Records, screens, workflow, controls and hand-offs

Accounts

RASP module · accounts

Purpose	The accounting source of truth: balanced, posted journal vouchers, a dated trial balance, receivables, receipts and reversal rather than edit.
Core records	Chart of accounts, vouchers/lines, posting date/status/reference, reversal links/reasons, customer receivables and receipt allocations.
User surface	Trial balance; Vouchers; Voucher detail. Dashboard values name the as-at date and whether pagination caps the visible count.
End-to-end flow	A domain prepares a balanced voucher; Accounts validates and posts it; a deferred database check re-sums at commit; invoice postings create receivables; receipts settle oldest first; errors are corrected by an opposite voucher linked to the original.
Enforced controls	Triple balance checks, append-only posted records, one allowed reversal-link update, idempotency, tenant RLS, dated reporting and synchronous posting for transaction-critical domains.
Inputs and outputs	Receives Sales invoices and Payroll postings; supplies Sales credit exposure, Employee Spend acknowledgements, RASP finance tools and the working-capital starting point.
Current boundary	General ledger, trial balance, receivables and receipts are live. Period close/reopen, payables, bank reconciliation and full financial statements are incomplete.
Primary code map	`apps/web/src/modules/accounts/manifest.ts · apps/api/src/modules/accounts/`

Employee Spend

RASP module · expenditure

Purpose	Controls employee and indirect spend before money leaves by reserving budget at request time and preserving policy, tax and receipt evidence.
Core records	Budgets and signed reservation movements, travel requests, claims/lines, attachments/extraction decisions, advances/refunds, indirect expenses, posting queue, duplicate/AI reports, TDS and ITC registers.
User surface	Claims; Claim detail. The API also covers budgets, receipts, advances, travel, indirect expense, posting and tax reports.
End-to-end flow	Check budget under a row lock; create claim/travel/expense; attach and optionally extract a receipt; submit to reserve in-approval budget; approve/reject with policy evidence; move reservation to committed/actual through signed rows; prepare an accounting posting.
Enforced controls	Atomic reservation ledger, no silent unbudgeted approval, mandatory override reason, duplicate receipt checks, human confirmation of extraction, input-credit and withholding gates, idempotency and approval separation.
Inputs and outputs	Uses People identity/grade, Organisation financial year, AI Operations for receipt extraction, and is designed to post to Accounts after acknowledgement.
Current boundary	Budget, claim, receipt and policy controls are live. Prepared spend postings are not yet wired end-to-end to the ledger, and cost-centre vocabularies need alignment.
Primary code map	`apps/web/src/modules/expenditure/manifest.ts · apps/api/src/modules/expenditure/`

How to read these module pages

“Live” means the records, service rules and user/API surface exist in this repository. It does not mean every surrounding enterprise process is complete. The boundary row names what remains illustrative, manual, simulate-driven or outside the MVP.

How RASP's modules connect

A module owns its records; the agent explains the combined evidence

People

RASP module · hrm

Purpose	Connects accountable employees, attendance and statutory payroll to the work and money records they create.
Core records	Employees, employment/grade/bank/statutory identifiers, punch events, attendance muster and regularisation, leave, payroll runs/payslips, dated statutory rates and posting references.
User surface	Employees; Employee detail; Muster; Leave; Statutory.
End-to-end flow	Capture punches; derive attendance days; hold incomplete days for regularisation; append corrections and replay the day; calculate deemed wages and dated EPF/ESI/professional-tax/withholding; keep preparer and approver separate; post payroll to Accounts in one transaction.
Enforced controls	Append-only punch correction, no one-punch-as-present shortcut, dated statutory tables, fail-loudly on missing rates, separation of payroll duties, encrypted sensitive fields and synchronous ledger posting.
Inputs and outputs	Supplies accountable people to workflow, Maintenance, Production and Spend; sends approved payroll postings to Accounts and workforce evidence to RASP.
Current boundary	Employee, attendance and payroll foundations are live. Recruitment, performance, learning depth, biometric-device transport and broader HR lifecycle are not complete.
Primary code map	`apps/web/src/modules/hrm/manifest.ts · apps/api/src/modules/hrm/`

Working Capital

RASP module · working-capital

Purpose	A decision workspace that explains cash coming in/out, stock cash, margins, a 13-week outlook, scenarios and finance-readiness evidence in simple language.
Core records	Current screen models for priorities, scenario cards and evidence gaps; live cash-position evidence comes from posted Accounts data and Agent OS capability outputs.
User surface	Overview; Money in; Money out; Stock holding cash; Margins; 13-week forecast; Scenarios; Finance readiness pack.
End-to-end flow	Start from posted ledger evidence; bring in customer commitments and stock holding; label assumptions; compare bounded scenarios; prioritise human follow-up; draft an evidence manifest; route the brief through HEXA verification and a human gate.
Enforced controls	Read/simulate/analyse/draft modes only, no payment or posting capability, source and assumption labelling, human approval in the mission and an explicit boundary note on every tool result.
Inputs and outputs	Combines Accounts, MICA sales commitments and SPAR stock evidence; publishes a review brief and governed finance work item through Agent OS.
Current boundary	The cash-position read and governed mission are live. Most workspace numbers and three finance tools are illustrative/evidence wrappers, not a complete forecasting, collections or lender-pack engine.
Primary code map	`apps/web/src/modules/working-capital/manifest.ts · apps/api/src/agent-os/capability-registry.service.ts`

Cross-module coordination

Accounts is the posted truth; Employee Spend and People create controlled obligations and postings; Working Capital turns the available evidence into reviewable priorities and scenarios. RASP must label the difference between a ledger fact, an assumption and an illustrative demo value.

Accounts → **Employee Spend** → **People** → **Working Capital** → **RASP** bounded evidence → **ONYX / person** decision

How RASP's world actually runs

The business pipeline, not the agent plumbing

Money moves in one direction only. A journal is never edited — it is reversed and re-posted — the budget counts money the moment somebody asks for it, and every statutory rate is a dated row rather than a number in the code.

1. **Post**
   Balance is checked three times over: in the calculation, as a column constraint, and again by a deferred trigger that re-sums the lines at commit. A voucher that does not balance cannot exist.

2. **Correct**
   There is no edit. A wrong voucher is reversed by an opposite one carrying the reason, so the mistake survives its own correction. The database permits exactly one update — writing the reversal link.

3. **Receive**
   An invoice becomes a receivable due on the invoice date plus the customer's credit days. Receipts settle oldest first, and the remainder is the exposure MICA's credit gate reads.

4. **Commit — the reservation ledger**
   Available = budgeted − actual − committed − in-approval. Money is counted when it is requested, not when it is spent, which is why two people cannot both spend the same last ₹50,000.

5. **Move the reservation**
   in_approval → committed on approval, committed → actual on the accounting acknowledgement — written as two signed rows, never an update. Summing the column IS the balance.

6. **Attend**
   Punches become attendance days. One punch is never silently counted as present; it is held for regularisation, and a correction appends fresh punches and replays the day rather than overwriting a figure.

7. **Pay**
   Attendance becomes deemed wages, then EPF, ESI, professional tax and withholding — each resolved from the rate in force at month end and stamped onto the payslip. Payroll posts to the ledger synchronously, in one transaction.

How much of this RASP can actually see

Of everything above, RASP can read exactly 5 things directly — journal vouchers, trial balance, 13-week scenario, review queue, draft manifest. The rest of this pipeline is context for judging what it reads; it is not data the agent can reach. Where a mission needs more, it asks the specialist who owns it or it says the evidence is missing.

What the code will not let anyone do

Enforced by constraints and locks, not by wording

Reversal, not correction

An append-only trigger makes editing a posted journal impossible rather than merely discouraged — and it fires for the schema owner too.

The reservation is taken under a row lock

The budget line is locked before availability is even read, inside the caller's transaction, so a crash between “money reserved” and “document created” rolls both back together.

An unbudgeted spend warns; it never passes silently

A missing budget line returns a warning with a zero availability and the words “this spend is unbudgeted” — not a quiet approval.

A blocked spend can be overridden, but never anonymously

An override without a written reason is refused outright: an override nobody can read afterwards is no control at all.

Some tax positions are not the software's to take

When a withholding threshold is crossed mid-year there are two defensible readings. The system computes BOTH, marks the document for finance review, and refuses to choose — because that is a tax position and it belongs to a person.

A missing statutory rate fails loudly

Payroll stops rather than guessing. A wrong provident-fund deduction is a debt with interest attached, so no rate is ever assumed.

> **Why this page matters more than the agent pages**
> An agent is only as trustworthy as the system it reads. Every rule here holds whether the request came from a person, a screen, a seeder or RASP — which is precisely why an agent can be given a read tool without also being given a way to do damage.

Where these rules are kept

```
apps/api/src/modules/accounts/accounts.service.ts
```

```
apps/api/src/modules/expenditure/budget.service.ts
```

```
apps/api/src/modules/hrm/payroll.service.ts
```

How RASP takes part in a mission

The engine controls order, parallelism and recovery

Person or signal → **ONYX** scope the mission → **RASP** Finance & People → **Capability registry** allow-listed tool → **Domain service** business rules → **Tenant data** + audit

1 · Scope

ONYX turns the goal into a run against a frozen graph version, with a fixed tenant, user, step budget and timeout.

2 · Authorise

Two checks, both required: the tool must be on RASP's allow-list, and the signed-in person must independently hold the permission.

3 · Execute

A registered domain service does the work under the same rules a screen would face. There is no general SQL doorway.

4 · Preserve

Inputs, outputs, events and a checkpoint after every wave, so the run can be explained or resumed.

An agent can narrow authority, never widen it

RASP is a specialist and delegates to nobody. Because the permission check is repeated against the requesting person, a mission can never reach data that person could not have opened themselves.

06 · EXACT CALLABLE SURFACE

Everything RASP can call

This table is the complete list — there is no other door

Capability	Mode	What comes back	Permission required	Needs approval
accounts.vouchers.read	Read	Journal vouchers	accounts.ledger.read	No
finance.cash-position.read	Read	Trial balance	accounts.ledger.read	No
finance.forecast.simulate	Simulate	13-week scenario	accounts.ledger.read	No
finance.collections.prioritise	Analyse	Review queue	accounts.ledger.read	No
finance.funding-pack.draft	Draft	Draft manifest	accounts.ledger.read	No
agent.action.dispatch	Execute	Governed work item	agentos.run.operate	Yes

Read · Analyse · Simulate

Return evidence or a reproducible view. Nothing in the business changes.

Draft

Produce reviewable content that is labelled a draft and can never present itself as an approved outcome.

Execute

The only side-effecting mode in the whole system, and the only one that requires an approved human gate above it.

Both locks must open

The agent's allow-list AND the person's permission. Either one failing is a refusal.

What “dispatch” does and does not do

It appends a governed work item naming the responsible specialist, the approval it came from and the exact payload. It does not change a sales order, a stock balance, a production order, a journal or a customer message — a person still does that work.

Where RASP actually appears

Node by node, from the registered graph definitions

Mission graph	What RASP does in it
Cross-functional readiness	Not involved
Nine-agent operating review	Reads vouchers, then the finance assessment
Controlled action mission	Reads, recommends, and dispatches the financial guardrail
Working Capital Review	Owns three of nine nodes — cash position, 13-week simulation, the analysis
QMS & Audit Readiness	Not involved
Managed Service Assurance	Not involved; validates financial effects only when requested

Reading this honestly

“Not involved” is not a limitation, it is the design. A mission asks only the specialists whose evidence it needs, and a specialist cannot add itself to a graph at runtime. The graph is written down, versioned and content-hashed before the run starts.

The shape every mission shares

Goal → **Parallel reads** → **Assessments** → **Join** → **HEXA verify** → **Human gate** → **Outcome**

08 · WORKED EXAMPLE

What the order actually did to the books

Real records from the running demo, not an illustration

Dispatching 28 of the 120 pumps raised INV-2627-00002 in the same transaction as the goods-out movement — the stock and the receivable cannot disagree, because they commit together or not at all.

Northstar paid ₹10 lakh against it. The balance is what the credit gate will read the next time this customer orders.

The quality engineer's trip to Northstar's works was budget-checked ON SUBMISSION against the FY 2026-27 travel budget, not after the money had gone. The three people who built the pumps were rostered, and their attendance came from punches rather than a spreadsheet.

> **The sentence to say out loud**
> “RASP contributes evidence from its own area, with the source records behind it and its limits stated. It does not claim an action is done until the system has recorded and verified it.”

Fair questions to ask while watching

- Which records is this answer standing on?
- Which permission was checked, and against whom?
- Is this a recorded fact, a simulation, a draft, or a completed result?
- Would this step have waited for a person?
- What happens if the service restarts halfway through?

09 · WHAT IS REAL

Live today, and deliberately not

The second column is the one worth reading

Working now

- Balanced, append-only double-entry with reversal
- Trial balance as at a posting date, and the receivables subledger
- Budget reservation across three buckets, under a row lock
- Deemed wages, EPF, ESI, professional tax and withholding from dated rate rows
- Payroll posted to the ledger synchronously, with preparer and approver kept separate

Not built — stated plainly

- The working-capital screens are illustrative, not live — the forecast, collection queue and funding pack carry no arithmetic behind them
- Of the four finance tools, only the cash-position read computes anything; the other three return evidence with a boundary note
- There is no way to close or reopen an accounting period, and no payables subledger
- Employee-spend postings are prepared but not wired through to the ledger — payroll's are
- The employee and budget cost-centre vocabularies do not overlap, so a claim can read as unbudgeted when a budget exists

Identity boundary	Verified token → tenant and actor
Authority boundary	Agent allow-list AND the person's permission
Action boundary	Side effects need an approved ancestor node
Data boundary	Domain service over a tenant-fenced database
Evidence boundary	Append-only runs, events, checkpoints and audit

Gaps are listed because a guide that hides them fails the first hour of technical due diligence. Every item on the right is either a roadmap decision or a known defect, and both are recorded in the project's own capability-gap register.

RASP on one page

For explaining the agent to somebody new

Its job

Explains the accounting truth, the cash position, what is already committed, and the payroll consequence behind an operating decision.

Speaks for

Accounts, Employee spend, People, Working capital

Takes in

Journals, receivables and receipts · Order, purchasing and stock pressure · Budget commitments and claims · Attendance and payroll outcomes

Gives back

The accounting position, as posted · A scenario clearly labelled as a scenario · A collection priority order · A finance work item or a reviewable draft

Can call

accounts.vouchers.read, finance.cash-position.read, finance.forecast.simulate, finance.collections.prioritise, finance.funding-pack.draft, agent.action.dispatch

Cannot do

The working-capital screens are illustrative, not live — the forecast, collection queue and funding pack carry no arithmetic behind them · Of the four finance tools, only the cash-position read computes anything; the other three return evidence with a boundary note · There is no way to close or reopen an accounting period, and no payables subledger

Words used on these pages

Agent	A named specialist with a fixed, allow-listed set of tools — not a process that can roam.
Capability	One registered operation with a declared mode, owner and required permission.
Mission graph	A versioned recipe: which steps run, which run together, where it waits for a person.
Checkpoint	A stored snapshot after each wave, used to explain a run and to resume it safely.
Governed work item	An approved, append-only assignment for a human team — not the business change itself.